

Laporan Milestone 3 Semantic Analysis

IF2224 TEORI BAHASA FORMAL DAN OTOMATA



Disusun oleh:

BST – brawlstars

Kevin Wirya Valerian 13524019

Ray Owen Martin 13524033

Emilio Justin 13524043

Philipp Hamara 13524101

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

Daftar Isi.....	2
Daftar Gambar.....	3
I. Landasan Teori.....	4
A. Semantic Analysis.....	4
B. Abstract Syntax Tree (AST).....	4
C. Decorated Abstract Syntax Tree (Decorated AST).....	6
D. Algoritma L-Attributed Grammar.....	7
II. Perancangan dan Implementasi.....	9
A. Revisi Milestone 2.....	9
B. Rancangan Tree.....	9
C. Implementasi Program.....	10
III. Pengujian.....	13
A. Pengujian 1.....	13
B. Pengujian 2.....	14
C. Pengujian 3.....	16
D. Pengujian 4.....	19
E. Pengujian 5.....	20
F. Pengujian 6.....	21
G. Pengujian 7.....	27
H. Pengujian 8.....	27
I. Pengujian 9.....	28
J. Pengujian 10.....	29
K. Pengujian 11.....	31
L. Pengujian 12.....	31
IV. Kesimpulan dan Saran.....	33
A. Kesimpulan.....	33
B. Saran.....	33
Referensi.....	34
Lampiran A. Link Penting.....	35

Daftar Gambar

I. Landasan Teori

A. Semantic Analysis

Dalam membuat suatu *compiler*, selain membutuhkan pengecekan syntax, kita perlu melakukan pengecekan makna (**semantic analysis**) dari program. Pengecekan tersebut terdiri atas beberapa hal, yaitu

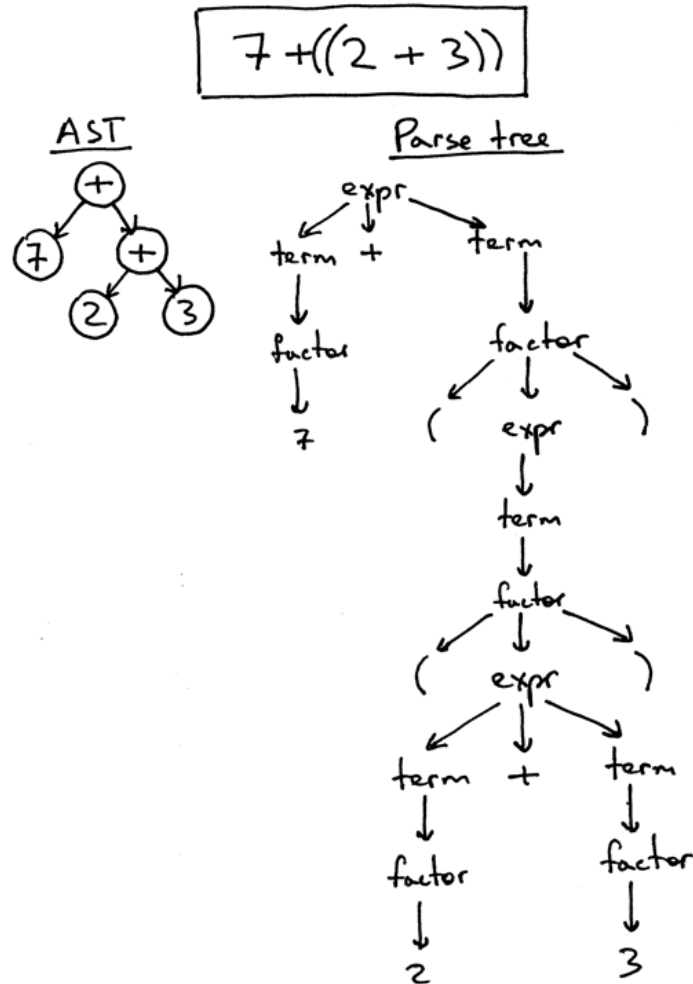
- Type Checking, untuk memastikan operator dan operand memiliki tipe yang kompatibel.
- Symbol table management, untuk memastikan setiap identifer telah dideklarasikan sebelum digunakan dan tidak dideklarasikan kembali dalam scope yang sama.
- Scope resolution, untuk menentukan validitas akses variabel berdasarkan hierarki blok kode.
- Control flow validation, untuk memastikan struktur alur kontrol yang logis.

Semantic analysis akan melakukan anotasi terhadap parse tree yang telah dihasilkan oleh parser dan menghasilkan sebuah decorated abstract syntax tree (AST). Anotasi berupa penambahan informasi, seperti tipe data dan identifer ke symbol table.

B. Abstract Syntax Tree (AST)

Secara sederhana, **Abstract Syntax Tree** (AST) merupakan versi efisien dari parse tree yang telah disaring. Jika parse tree berfungsi untuk memvalidasi struktur sintaksis berdasarkan aturan tata bahasa (grammar), AST membuang detail administratif tersebut dan hanya menyisahkan informasi esensial. Berikut adalah poin-poin perubahannya.

- AST mengeliminasi elemen grammar yang berlebihan untuk meringankan beban komputasi pada tahapan proses selanjutnya.
- Dari yang semula berfokus pada sintaks, kini program beralih ke semantik.
- Struktur AST menitikberatkan pada keterkaitan antara operator dan operand.



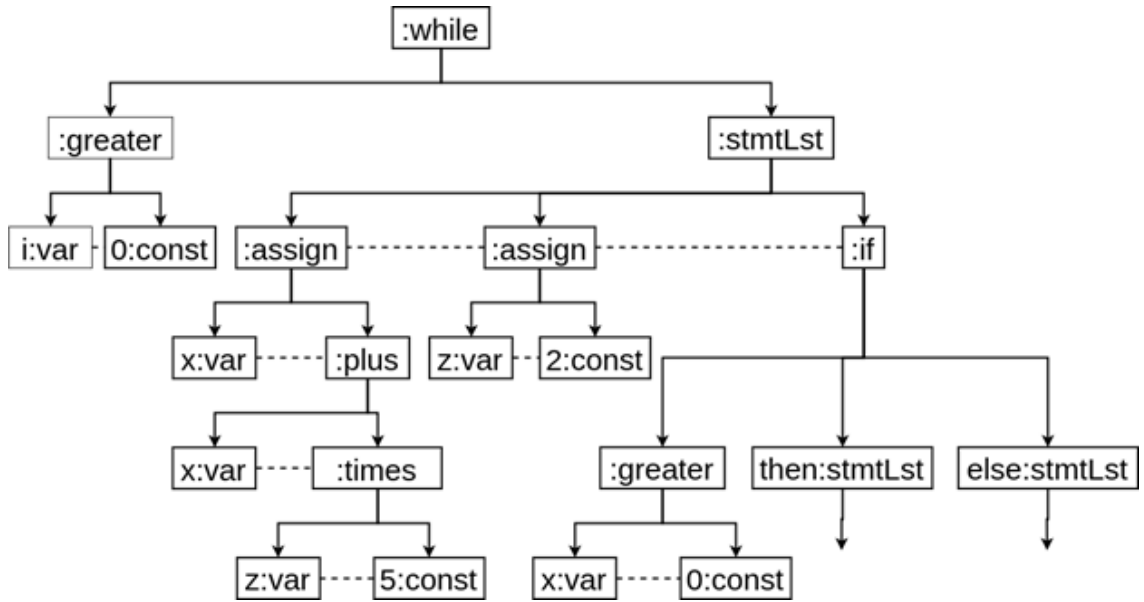
Gambar Parse Tree dan Abstract Syntax Tree (AST) untuk ekspresi matematika

(Sumber: <https://ruslanspivak.com/lsbasi-part7>)

Sebagai ilustrasi, untuk potongan kode berikut.

```
/* TIDAK DIPASTIKAN SESUAI BAHASA ARION */
while (i > 0) {
    x = x + z * 5;
    z = 2;
    if (x > 0) then { ... } else { ... }
}
```

Kode tersebut akan menghasilkan Abstract Syntax Tree:



Gambar Contoh Abstract Syntax Tree (AST)
(Sumber: <https://nus-cs3203.github.io>)

C. Decorated Abstract Syntax Tree (Decorated AST)

Dalam tahap semantic analysis, *abstract syntax tree* (AST) akan diperkaya dengan informasi tambahan melalui proses anotasi. Hasil akhirnya disebut sebagai decorated AST. Secara garis besar, proses ini memberikan logika pada syntax tree agar compiler memahami makna di balik struktur kode. Anotasi tersebut mencakup identitas variabel, validasi tipe data, hingga batasan scope variabel.

Berikut adalah komponen kunci dalam pembentukan Decorated AST:

1. Attributed Grammar

Grammar yang dilengkapi dengan atribut dan aturan semantik.

2. Symbol Table

Struktur data yang minimal menyimpan informasi nama identifier, tipe datanya dan scopenya. Symbol table menggunakan struktur data *stack*.

3. Visitor / Visit Functions

Untuk setiap jenis node akan memiliki fungsi `visit_<node>` yang akan:

- i. Memanggil visit pada child-nya
- ii. Mengakses atau memperbarui symbol table
- iii. Menghitung atribut/tipe semantik
- iv. Menambahkan anotasi pada node

D. Algoritma L-Attributed Grammar

Dalam pengembangan compiler Arion, algoritma L-Attributed Grammar digunakan sebagai dasar untuk melakukan analisis semantik dan membangun Decorated AST.

L-Attributed Grammar adalah jenis Syntax-Directed Definition (SDD) yang membatasi ketergantungan atribut agar informasi dapat mengalir secara efisien dalam sekali penelusuran. Nama "L" merujuk pada aliran informasi yang bergerak dari kiri ke kanan. Secara teknis, sebuah atribut pada simbol tata bahasa dikatakan L-attributed jika nilainya hanya bergantung pada

1. Inherited attributes dari simbol *parent*.
2. Inherited atau synthesized attributes dari simbol *left siblings*.

L-Attributed Grammar diterapkan melalui fungsi **visit** (semantic visitor) yang menelusuri setiap node pada parse tree secara *top-down* dan *depth-first*. Aturan semantik disisipkan langsung ke dalam aturan produksi tata bahasa. Setiap kali visitor mengunjungi node tertentu, semantic action akan dieksekusi untuk mengevaluasi nilai atribut. Kemudian, *inherited attributes* akan digunakan untuk meneruskan informasi dari atas ke bawah, seperti meneruskan informasi lexical level atau scope ke blok di dalamnya. Di sisi lain, *synthesized attributes* digunakan untuk menaikkan informasi dari bawah ke atas, seperti menentukan tipe data akhir dari sebuah ekspresi matematika berdasarkan tipe operand-nya.

Algoritma ini sangat krusial dalam dua proses utama:

1. **Konversi Parse Tree ke AST**

Menghilangkan detail sintaksis (seperti tanda kurung atau keyword BEGIN/END) dan membentuk struktur yang hanya berfokus pada hubungan operator dan operand.

2. **Dekorasi AST**

Melakukan *type checking* dan *scope resolution* sambil menelusuri pohon, algoritma ini mengelola *symbol table* dan memberikan anotasi berupa tipe data pada setiap node.

II. Perancangan dan Implementasi

A. Revisi Milestone 2

Dalam pengerjaan Milestone 2, dilakukan revisi pada parser. Pada milestone sebelumnya, comment tidak terdeteksi pada parser, oleh karena itu ditambahkan agar comment dapat dideteksi tetapi tidak dijadikan sebagai node pada parse tree. Selain itu, empty statement juga disempurnakan sehingga dapat diterima oleh parse tree. Yang terakhir dan yang paling utama, node untuk while-loop dan for-loop sudah menggunakan compound statement untuk statement yang perlu dilakukan, jadi dosy diikuti compound-statement. Akan tetapi untuk perubahan ini, sintaks input while dan for diubah sehingga harus ada begin dan end untuk menandakan awal dan akhir compound statement setelah dosy.

B. Rancangan Tree

Selayaknya AST yang merupakan ringkasan dari token-token pada parse tree, AST yang diimplementasikan di milestone ini mengartikan token-token tadi menjadi yang lebih bermakna, dari setiap token termasuk kurung dan tanda baca menjadi beberapa kelompok seperti:

- Program, berisi name dan blocknode. Name menunjukkan nama program, sementara blocknode dijelaskan di bawah ini.
- BlockNode, yaitu node utama di AST. BlockNode memiliki declaration dan statement yang dijelaskan di bawah ini. Meskipun hasil output menunjukkan BlockNode hanya sebagai kumpulan statement, tetapi secara program maknanya adalah gabungan declaration dan statement
- Declaration, yaitu deklarasi semua variable, fungsi, dan prosedur pada program yang terdapat di bawah nama program dan di atas kumpulan statement pada file input. Bagian ini yang memperkenalkan variabel, fungsi dan prosedur. Declaration direpresentasikan sebagai DeclNode. Declaration memiliki beberapa turunan:
 - ConstDeclNode : deklarasi konstanta
 - TypeDeclNode : deklarasi tipe baru
 - VarDeclNode : deklarasi variabel
 - SubprogramDeclNode : deklarasi prosedur dan fungsi.

Bagian ini memiliki nama dan BlockNode. Kalau berupa fungsi, akan memiliki return type. Subprogram juga memiliki komponen yang tidak harus ada, yaitu parameter, selayaknya prosedur dan fungsi pada umumnya. Ini direpresentasikan sebagai ParamNode.

- Statement, yaitu perintah yang dieksekusi saat program berjalan. Statement direpresentasikan sebagai StmtNode yang memiliki beberapa turunan berikut:
 - AssignStmtNode : bagian assign suatu variabel dengan nilai tertentu
 - CompoundStmtNode: daftar statement
 - IfStmtNode : block kondisional yang ditandai if
 - CaseStmtNode : block kondisional bercabang yang ditandai case
 - WhileStmtNode : block loop dengan while
 - RepeatStmtNode : block loop dengan repeat-until
 - ForStmtNode : block loop dengan for
 - ProcCallStmtNode : bagian pemanggilan prosedur (pemanggilan fungsi tidak di-handle di sini, melainkan di expression)
 - EmptyStmtNode : statement kosong
- Expression, yaitu ekspresi; sesuatu yang menghasilkan nilai. Expression direpresentasikan dengan ExprNode yang memiliki beberapa turunan berikut:
 - LiteralNode : nilai literal, yang ditulis langsung pada bahasa program Arion
 - VarAccessNode : akses variabel
 - BinOpNode : operasi biner
 - UnaryOpNode : operasi uner
 - FuncCallNode : pemanggilan fungsi
- Type, yaitu tipe data. Type direpresentasikan dengan TypeNode yang memiliki beberapa turunan berikut:
 - NamedTypeNode : tipe data primitif
 - RangeNode : subrange
 - ArrayTypeNode : tipe array, dengan tipe indeks dan element
 - EnumTypeNode : tipe enumerasi
 - RecordTypeNode : untuk record

C. Implementasi Program

1. Implementasi Semantic Analysis

Implementasi semantic analysis dilakukan dengan menggunakan AST dan visitor. Setelah parser menghasilkan parse tree, struktur tersebut diubah menjadi AST oleh ASTBuilder. AST lalu dianalisis untuk diperiksa konsistensi tipe, deklarasi, scope, dan berbagai aturan semantik lain. Implementasinya terlihat di file ast_builder.cpp, ast.cpp, dan semantic.cpp.

Cara kerja utama program ini adalah SemanticVisitor mengunjungi semua node. Jika merupakan deklarasi, maka akan dimasukkan ke symbol table dan block table. Jika berupa VarAccessNode, maka akan dilakukan lookup untuk mendapatkan tipe dan scope. Jika berupa operasi ekspresi, maka divalidasi dulu tipenya lalu hasil akhirnya dicetak sebagai decorated AST.

2. Pembentukan AST

ASTBuilder bertanggung jawab dalam membuat AST. Secara umum, pembuatan AST akan traversal parse tree dan membangun node AST yang lebih ringkas dan bermakna dibandingkan parse tree. Beberapa fungsi yang digunakan:

- build() : menghasilkan ProgramNode dari <program>. ProgramNode ini nantinya akan berisi BlockNode
- buildDeclarationPart() : membangun bagian deklarasi
- buildStatement() : memetakan statement ke node turunan StmtNode.
- buildExpression() : membangun ekspresi

3. Struktur AST

Struktur AST diimplementasikan seperti yang dijelaskan di bagian [B. Rancangan Tree](#)

4. Visitor Semantic

AST diproses dengan pola visitor, yakni tiap node memiliki procedure accept(), sebagaimana diimplementasikan di ast.cpp, sementara SemanticVisitor mengimplementasikan procedure visit() untuk semua node, sebagaimana diimplementasikan di semantic.cpp. Dilakukan beberapa pengecekan, contohnya seperti identifier harus ada di symbol table, tiap ekspresi harus kompatibel saat assignment dan diberikan operasi, dan seterusnya.

5. Symbol Table

Symbol Table menyimpan informasi semua identifier. Symbol table diimplementasikan di symtab.cpp, sementara struktur tabelnya dideklarasikan di symtab.hpp. Atribut tabel (yang diimplementasikan sebagai TabEntry) adalah sebagai berikut:

- name : nama identifier
- obj : kelas objek, seperti program, variable, function, dan lain-lain
- type : tipe dasar identifier, seperti 1 untuk integer, 2 untuk real, dan seterusnya
- ref : referensi ke tabel lain, seperti ke array table dan block table

- nrm : jumlah nama atau parameter yang terkait dalam deklarasi
- lev : level scope, 0 untuk global, 1 untuk subprogram, dan seterusnya
- adr : offset dalam block
- link : pointer ke entri sebelumnya dalam scope yang sama

6. Block Table

Block Table menyimpan informasi setiap scope (block). Block table diimplementasikan di `symtab.cpp`, sementara struktur tabelnya dideklarasikan di `symtab.hpp`. Atribut tabel (yang diimplementasikan sebagai `BTabEntry`) adalah sebagai berikut:

- last : entri terakhir di block
- lpar : indeks parameter pertama untuk subprogram jika ada
- psze : jumlah/ukuran parameter di dalam block
- vsze : jumlah/ukuran variabel lokal dalam block

7. Array Table

Array Table menyimpan informasi semua identifier. Array table diimplementasikan di `symtab.cpp`, sementara struktur tabelnya dideklarasikan di `symtab.hpp`. Atribut tabel (yang diimplementasikan sebagai `ATabEntry`) adalah sebagai berikut:

- xtyp : tipe indeks array
- etyp : tipe elemen array
- eref : referensi tambahan jika ada
- low : batas bawah indeks array
- high : batas atas indeks array
- elsz : ukuran satu elemen
- size : ukuran total array

8. Error Handling

Setiap error dihandle dengan prosedur `addError()` yang hanya menambahkan error ke atribut `errors` pada `SemanticVisitor`.

III. Pengujian

A. Pengujian 1

Input

```

program Hello;

var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.

```

Output

```

=== SEMANTIC ANALYSIS BERHASIL (TIDAK ADA ERROR) ===

=== SYMBOL TABLE (tab) ===
idx  id          obj          type  ref  nrm  lev  adr  link
-----
...  (reserved words 0-32)
33   Hello       program    0     0    1    0    0    0
34   a           variable   1     0    1    0    0    33
35   b           variable   1     0    1    0    1    34
36   writeln     procedure  0     0    1    0    0    35

=== BLOCK TABLE (btab) ===
idx  last  lpar  psze  vsze
-----
0     36    0     0     2      <- global block (program)
1      0    0     0     0

=== ARRAY TABLE (atab) ===
(kosong karena tidak ada array)

=== DECORATED AST ===
└ ProgramNode(name: 'Hello') → tab_index:33, lev:0
  └ Declarations
    └ VarDecl('a') → tab_index:34, type:integer, lev:0
    └ VarDecl('b') → tab_index:35, type:integer, lev:0
  └ Block → block_index:1, lev:1
    └ Assign('a' := 5) → type:void
      └ target 'a' → tab_index:34, type:integer
      └ value 5 → type:integer

```

```

└─ Assign('b' := a+10) → type:void
   └─ target 'b' → tab_index:35, type:integer
      value
      └─ BinOp '+' → type:integer
         └─ 'a' → tab_index:34, type:integer
            10 → type:integer
   writeln(...) → predefined, tab_index:36
   └─ 'Result = ' → type:string
      └─ 'b' → tab_index:35, type:integer

```

B. Pengujian 2

Input

```

program Subprograms;

var
    i, total: integer;

procedure AddToTotal(x: integer; y: integer);
begin
    total := total + x + y;
end;

function NextVal(v: integer): integer;
begin
    NextVal := v + 1;
end;

begin
    i := 1;
    total := 0;
    AddToTotal(i, NextVal(i));
    if total > 0 then
        total := total + 1
    else
        total := total - 1;
    end.

```

Output

```

=== SEMANTIC ANALYSIS BERHASIL (TIDAK ADA ERROR) ===

=== SYMBOL TABLE (tab) ===
idx  id          obj          type  ref  nrm  lev  adr  link
-----
...  (reserved words 0-32)
33   Subprograms  program    0     0    1    0    0    0

```

```

34  i          variable  1    0    1    0    0    33
35  total      variable  1    0    1    0    1    34
36  AddToTotal procedure  0    1    1    0    0    35
37  x          variable  1    0    0    1    0    0
38  y          variable  1    0    0    1    1    37
39  NextVal    function  1    2    1    0    0    36
40  v          variable  1    0    0    1    0    0

=== BLOCK TABLE (btab) ===
idx  last  lpar  psze  vsze
-----
0    39    0     0     2      <- global block (program)
1    38    38     2     0
2    40    40     1     0
3     0     0     0     0

=== ARRAY TABLE (atab) ===
(kosong karena tidak ada array)

=== DECORATED AST ===
└ ProgramNode(name: 'Subprograms') → tab_index:33, lev:0
  └ Declarations
    └ VarDecl('i') → tab_index:34, type:integer, lev:0
    └ VarDecl('total') → tab_index:35, type:integer, lev:0
    └ ProcDecl('AddToTotal') → tab_index:36, lev:0
      └ Param(x) → type:integer
      └ Param(y) → type:integer
      └ Block → block_index:1, lev:1
        └ CompoundStmt
          └ Assign('total' := ...+y) → type:void
            └ target 'total' → tab_index:35, type:integer
            └ value
              └ BinOp '+' → type:integer
                └ BinOp '+' → type:integer
                  └ 'total' → tab_index:35, type:integer
                  └ 'x' → tab_index:37, type:integer
                └ 'y' → tab_index:38, type:integer
      └ FuncDecl('NextVal') : integer → tab_index:39, lev:0
        └ Param(v) → type:integer
        └ Block → block_index:2, lev:1
          └ CompoundStmt
            └ Assign('NextVal' := v+1) → type:void
              └ target 'NextVal' → tab_index:39, type:integer
              └ value
                └ BinOp '+' → type:integer
                  └ 'v' → tab_index:40, type:integer
                  └ 1 → type:integer
        └ Block → block_index:3, lev:1
          └ Assign('i' := 1) → type:void
            └ target 'i' → tab_index:34, type:integer
            └ value 1 → type:integer

```

```

└─ Assign('total' := 0) → type:void
  └─ target 'total' → tab_index:35, type:integer
    └─ value 0 → type:integer
└─ AddToTotal(...) → predefined, tab_index:36
  └─ 'i' → tab_index:34, type:integer
    └─ FuncCall('NextVal') → tab_index:39, type:integer, lev:0
      └─ 'i' → tab_index:34, type:integer
└─ IfStmt
  └─ condition
    └─ BinOp '>' → type:boolean
      └─ 'total' → tab_index:35, type:integer
        └─ 0 → type:integer
  └─ then
    └─ Assign('total' := total+1) → type:void
      └─ target 'total' → tab_index:35, type:integer
        └─ value
          └─ BinOp '+' → type:integer
            └─ 'total' → tab_index:35, type:integer
              └─ 1 → type:integer
  └─ else
    └─ Assign('total' := total-1) → type:void
      └─ target 'total' → tab_index:35, type:integer
        └─ value
          └─ BinOp '-' → type:integer
            └─ 'total' → tab_index:35, type:integer
              └─ 1 → type:integer

```

C. Pengujian 3

Input

```

program ControlFlow;

var
  i, x, y: integer;

begin
  x := 0;
  for i := 1 to 3 do
    begin
      x := x + i;
      y := 1;
    end;

  while x > 0 do
    begin
      x := x - 1;
      y := 0;
    end;
  end;
end;

```



```

    end;

    repeat
        x := x + 2;
    until x > 1;

    if x > 1 then
        x := 5
    else
        x := 6;

    case x of
        1: x := 10;
        2, 3: x := 20;
    end;
end.

```

Output

=== SEMANTIC ANALYSIS BERHASIL (TIDAK ADA ERROR) ===

=== SYMBOL TABLE (tab) ===

idx	id	obj	type	ref	nrm	lev	adr	link
...	(reserved words 0-32)							
33	ControlFlow	program	0	0	1	0	0	0
34	i	variable	1	0	1	0	0	33
35	x	variable	1	0	1	0	1	34
36	y	variable	1	0	1	0	2	35

=== BLOCK TABLE (btab) ===

idx	last	lpar	psize	vsze	
0	36	0	0	3	<- global block (program)
1	0	0	0	0	

=== ARRAY TABLE (atab) ===

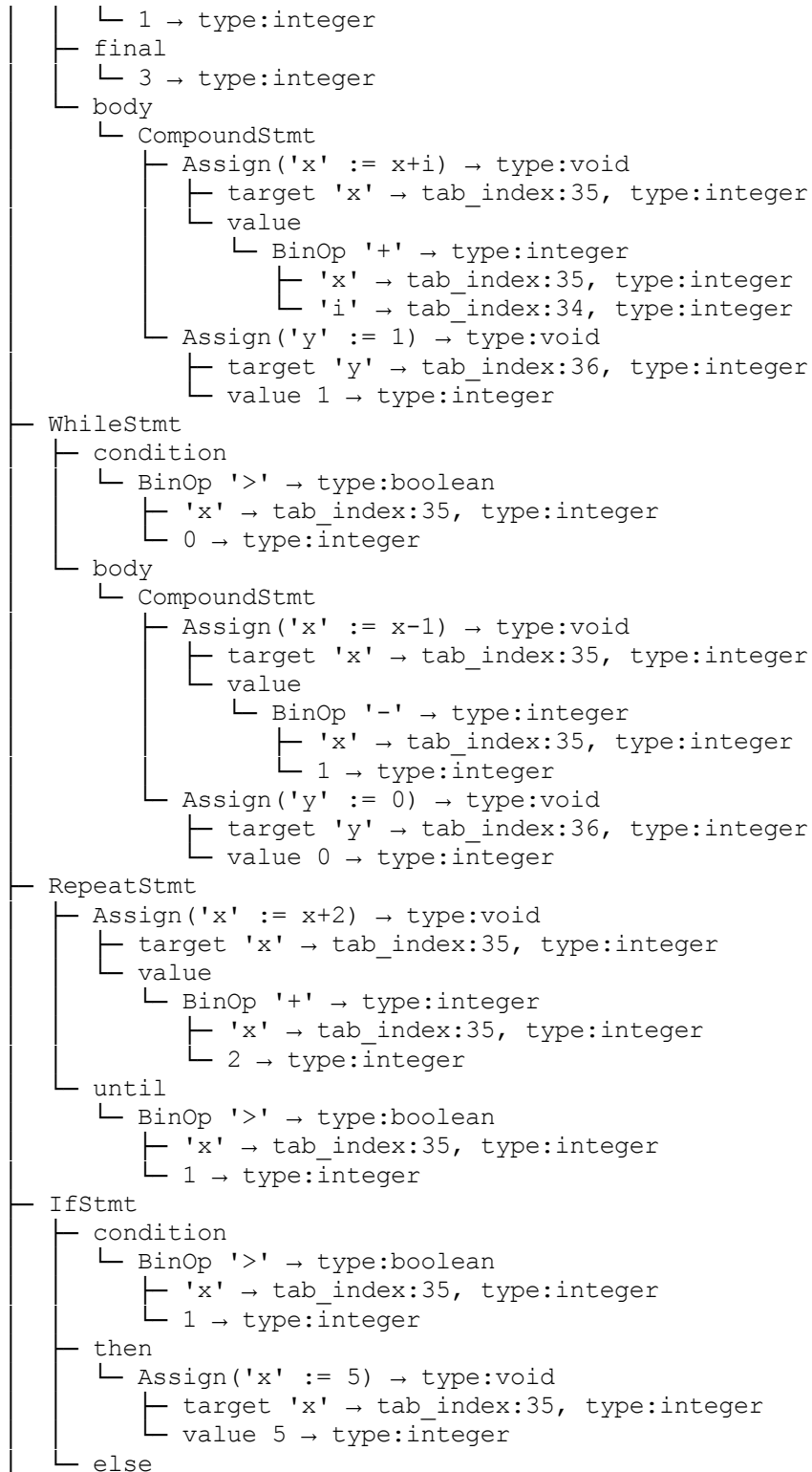
(kosong karena tidak ada array)

=== DECORATED AST ===

```

└─ ProgramNode(name: 'ControlFlow') → tab_index:33, lev:0
    └─ Declarations
        └─ VarDecl('i') → tab_index:34, type:integer, lev:0
            └─ VarDecl('x') → tab_index:35, type:integer, lev:0
                └─ VarDecl('y') → tab_index:36, type:integer, lev:0
                    └─ Block → block_index:1, lev:1
                        └─ Assign('x' := 0) → type:void
                            └─ target 'x' → tab_index:35, type:integer
                                └─ value 0 → type:integer
                            └─ ForStmt('i', tosy) → tab_index:34, lev:0
                                └─ init

```



```

└─ Assign('x' := 6) → type:void
   └─ target 'x' → tab_index:35, type:integer
      └─ value 6 → type:integer
└─ CaseStmt
   └─ expr
      └─ 'x' → tab_index:35, type:integer
   └─ CaseBlock
      └─ const: 1
         └─ Assign('x' := 10) → type:void
            └─ target 'x' → tab_index:35, type:integer
               └─ value 10 → type:integer

```

D. Pengujian 4

Input

```

program ArraysRecords;

var
  m: array[1..2] of array[1..2] of integer;
  p: record
    age: integer;
    name: string
  end;

begin
  m[1][2] := 7;
  p.age := m[1][2];
  p.name := 'abc';
end.

```

Output

```
=== SEMANTIC ANALYSIS BERHASIL (TIDAK ADA ERROR) ===
```

```
=== SYMBOL TABLE (tab) ===
```

idx	id	obj	type	ref	nrm	lev	adr	link

...	(reserved words 0-32)							
33	ArraysRecords	program	0	0	1	0	0	0
34	m	variable	0	0	1	0	0	33
35	age	field	1	0	1	0	0	34
36	name	field	5	0	1	0	0	35
37	p	variable	0	1	1	0	1	36

```
=== BLOCK TABLE (btab) ===
```

idx	last	lpar	psize	vsze
-----	------	------	-------	------

```

-----
0      37      0      0      2      <- global block (program)
1      36      0      0      0
2      0       0      0      0

=== ARRAY TABLE (atab) ===
idx  xtyp  etyp  eref  low  high  elsz  size
-----
0     1     1     0     0    0     1     1

=== DECORATED AST ===
└ ProgramNode(name: 'ArraysRecords') → tab_index:33, lev:0
  └ Declarations
    └ VarDecl('m') → tab_index:34, type:array, lev:0
    └ VarDecl('p') → tab_index:37, type:record, lev:0
  └ Block → block_index:2, lev:1
    └ Assign('m' := 7) → type:void
      └ target 'm' → tab_index:34, type:integer
      └ value 7 → type:integer
    └ Assign('p' := m) → type:void
      └ target 'p' → tab_index:37, type:integer
      └ value 'm' → tab_index:34, type:integer
    └ Assign('p' := 'abc') → type:void
      └ target 'p' → tab_index:37, type:string
      └ value 'abc' → type:string

```

E. Pengujian 5

Input

```

program Minimal;
begin
end.

```

Output

```

=== SEMANTIC ANALYSIS BERHASIL (TIDAK ADA ERROR) ===

=== SYMBOL TABLE (tab) ===
idx  id          obj          type  ref  nrm  lev  adr  link
-----
...  (reserved words 0-32)
33   Minimal      program      0     0    1    0    0    0

=== BLOCK TABLE (btab) ===
idx  last  lpar  psze  vsze
-----
0     33    0     0     0      <- global block (program)
1      0    0     0     0

```

```
=== ARRAY TABLE (atab) ===  
(kosong karena tidak ada array)  
  
=== DECORATED AST ===  
└ ProgramNode(name: 'Minimal') → tab_index:33, lev:0  
  └ Block → block_index:1, lev:1
```

F. Pengujian 6

Input

```
program Everything;  
  
var  
  i, j: integer;  
  x: real;  
  ch: char;  
  e: (A, B, C);  
  r: 1..3;  
  mat: array[1..3] of array[1..3] of integer;  
  rec: record  
    age: integer;  
    code: integer  
  end;  
  
procedure Update(a, b: integer; v: array[1..3] of integer);  
begin  
  v[1] := a + b;  
  v[2] := (a * 2) div 2;  
  v[3] := (v[1] + v[2]) mod 3;  
end;  
  
function NextVal(n: integer): integer;  
begin  
  NextVal := n + 1;  
end;  
  
begin  
  i := 1;  
  j := -i + 2 * 3 div 2 mod 2;  
  x := 6 / 2;  
  ch := 'z';  
  mat[1,2] := j;  
  rec.age := mat[1][2];  
  rec.code := NextVal(rec.age);  
  
  if not (j <= 0) or (i <> 0) then
```

```

    j := j + 1
else
    j := j - 1;

case j of
    1: j := 10;
    2, 3: j := 20;
end;

while (j > 0) and (i < 10) do
begin
    j := j - 1;
end;

repeat
    j := j + 1;
until j >= 2;

for i := 1 to 3 do
begin
    Update(i, j, mat[1]);
end;

for i := 3 downto 1 do
begin
    j := j + i;
end;

Update(i, j, mat[2]);
writeln('done', j, x, ch);
end.

```

Output

```
=== SEMANTIC ANALYSIS BERHASIL (TIDAK ADA ERROR) ===
```

```
=== SYMBOL TABLE (tab) ===
```

idx	id	obj	type	ref	nrm	lev	adr	link

...	(reserved words 0-32)							
33	Everything	program	0	0	1	0	0	0
34	i	variable	1	0	1	0	0	33
35	j	variable	1	0	1	0	1	34
36	x	variable	2	0	1	0	2	35
37	ch	variable	4	0	1	0	3	36
38	e	variable	0	0	1	0	4	37
39	r	variable	0	0	1	0	5	38
40	mat	variable	0	0	1	0	6	39
41	age	field	1	0	1	0	0	40
42	code	field	1	0	1	0	0	41
43	rec	variable	0	1	1	0	7	42

44	Update	procedure	0	2	1	0	0	43
45	a	variable	1	0	0	1	0	0
46	b	variable	1	0	0	1	1	45
47	v	variable	0	0	0	1	2	46
48	NextVal	function	1	3	1	0	0	44
49	n	variable	1	0	0	1	0	0
50	writeln	procedure	0	0	1	0	0	48


```

=== BLOCK TABLE (btab) ===
idx  last  lpar  psze  vsze
-----
0    50    0     0     8      <- global block (program)
1    42    0     0     0
2    47    47    3     0
3    49    49    1     0
4     0     0     0     0

```



```

=== ARRAY TABLE (atab) ===
idx  xtyp  etyp  eref  low  high  elsz  size
-----
0     1     1     0     0    0     1     1

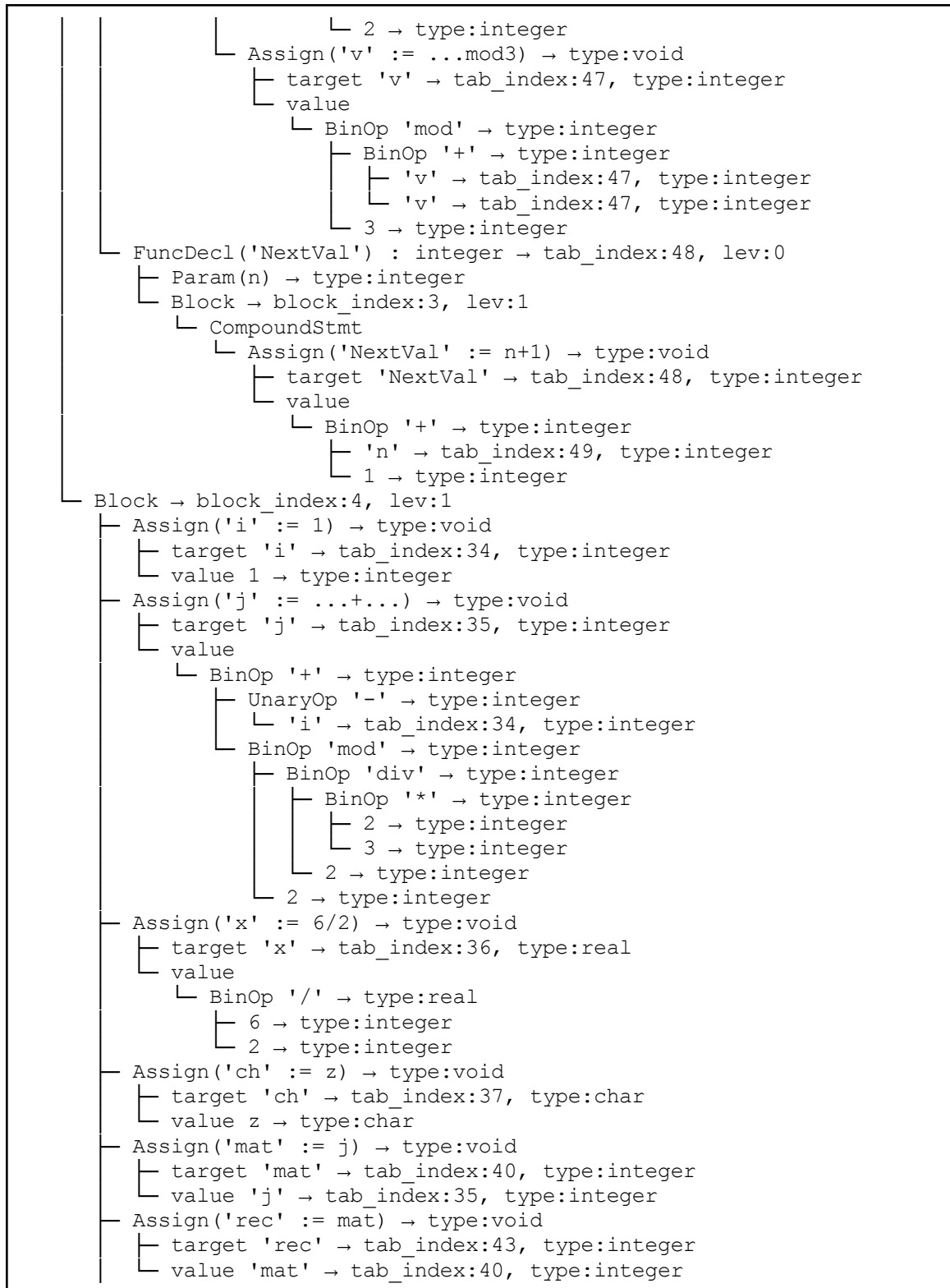
```



```

=== DECORATED AST ===
└─ ProgramNode(name: 'Everything') → tab_index:33, lev:0
    └─ Declarations
        └─ VarDecl('i') → tab_index:34, type:integer, lev:0
        └─ VarDecl('j') → tab_index:35, type:integer, lev:0
        └─ VarDecl('x') → tab_index:36, type:real, lev:0
        └─ VarDecl('ch') → tab_index:37, type:char, lev:0
        └─ VarDecl('e') → tab_index:38, type:unknown, lev:0
        └─ VarDecl('r') → tab_index:39, type:unknown, lev:0
        └─ VarDecl('mat') → tab_index:40, type:array, lev:0
        └─ VarDecl('rec') → tab_index:43, type:record, lev:0
        └─ ProcDecl('Update') → tab_index:44, lev:0
            └─ Param(a, b) → type:integer
            └─ Param(v) → type:unknown
            └─ Block → block_index:2, lev:1
                └─ CompoundStmt
                    └─ Assign('v' := a+b) → type:void
                        └─ target 'v' → tab_index:47, type:integer
                        └─ value
                            └─ BinOp '+' → type:integer
                                └─ 'a' → tab_index:45, type:integer
                                └─ 'b' → tab_index:46, type:integer
                    └─ Assign('v' := ...div2) → type:void
                        └─ target 'v' → tab_index:47, type:integer
                        └─ value
                            └─ BinOp 'div' → type:integer
                                └─ BinOp '*' → type:integer
                                    └─ 'a' → tab_index:45, type:integer
                                    └─ 2 → type:integer

```




```

└─ Assign('rec' := ) → type:void
  └─ target 'rec' → tab_index:43, type:integer
    └─ value
      └─ FuncCall('NextVal') → tab_index:48, type:integer, lev:0
        └─ 'rec' → tab_index:43, type:integer
└─ IfStmt
  └─ condition
    └─ BinOp 'or' → type:boolean
      └─ UnaryOp 'not' → type:boolean
        └─ BinOp '<=' → type:boolean
          └─ 'j' → tab_index:35, type:integer
            └─ 0 → type:integer
      └─ BinOp '<>' → type:boolean
        └─ 'i' → tab_index:34, type:integer
          └─ 0 → type:integer
  └─ then
    └─ Assign('j' := j+1) → type:void
      └─ target 'j' → tab_index:35, type:integer
        └─ value
          └─ BinOp '+' → type:integer
            └─ 'j' → tab_index:35, type:integer
              └─ 1 → type:integer
  └─ else
    └─ Assign('j' := j-1) → type:void
      └─ target 'j' → tab_index:35, type:integer
        └─ value
          └─ BinOp '-' → type:integer
            └─ 'j' → tab_index:35, type:integer
              └─ 1 → type:integer
└─ CaseStmt
  └─ expr
    └─ 'j' → tab_index:35, type:integer
  └─ CaseBlock
    └─ const: 1
      └─ Assign('j' := 10) → type:void
        └─ target 'j' → tab_index:35, type:integer
          └─ value 10 → type:integer
└─ WhileStmt
  └─ condition
    └─ BinOp 'and' → type:boolean
      └─ BinOp '>' → type:boolean
        └─ 'j' → tab_index:35, type:integer
          └─ 0 → type:integer
      └─ BinOp '<' → type:boolean
        └─ 'i' → tab_index:34, type:integer
          └─ 10 → type:integer
  └─ body
    └─ CompoundStmt
      └─ Assign('j' := j-1) → type:void
        └─ target 'j' → tab_index:35, type:integer
          └─ value

```

```

└─ BinOp '-' → type:integer
  └─ 'j' → tab_index:35, type:integer
    └─ 1 → type:integer
RepeatStmt
└─ Assign('j' := j+1) → type:void
  └─ target 'j' → tab_index:35, type:integer
    └─ value
      └─ BinOp '+' → type:integer
        └─ 'j' → tab_index:35, type:integer
          └─ 1 → type:integer
until
└─ BinOp '>=' → type:boolean
  └─ 'j' → tab_index:35, type:integer
    └─ 2 → type:integer
ForStmt('i', tosy) → tab_index:34, lev:0
└─ init
  └─ 1 → type:integer
└─ final
  └─ 3 → type:integer
└─ body
  └─ CompoundStmt
    └─ Update(...) → predefined, tab_index:44
      └─ 'i' → tab_index:34, type:integer
        └─ 'j' → tab_index:35, type:integer
          └─ 'mat' → tab_index:40, type:integer
ForStmt('i', downtosy) → tab_index:34, lev:0
└─ init
  └─ 3 → type:integer
└─ final
  └─ 1 → type:integer
└─ body
  └─ CompoundStmt
    └─ Assign('j' := j+i) → type:void
      └─ target 'j' → tab_index:35, type:integer
        └─ value
          └─ BinOp '+' → type:integer
            └─ 'j' → tab_index:35, type:integer
              └─ 'i' → tab_index:34, type:integer
Update(...) → predefined, tab_index:44
└─ 'i' → tab_index:34, type:integer
└─ 'j' → tab_index:35, type:integer
└─ 'mat' → tab_index:40, type:integer
writeln(...) → predefined, tab_index:50
└─ 'done' → type:string
└─ 'j' → tab_index:35, type:integer
└─ 'x' → tab_index:36, type:real
└─ 'ch' → tab_index:37, type:char

```

G. Pengujian 7

Input

```
program TesKosong;

begin
; {ini komentar}
end.
```

Output

```
=== SEMANTIC ANALYSIS BERHASIL (TIDAK ADA ERROR) ===

=== SYMBOL TABLE (tab) ===
idx  id              obj              type  ref  nrm  lev  adr  link
-----
...  (reserved words 0-32)
33   TesKosong       program      0     0    1    0    0    0

=== BLOCK TABLE (btab) ===
idx  last  lpar  psze  vsze
-----
0     33    0     0     0      <- global block (program)
1      0    0     0     0

=== ARRAY TABLE (atab) ===
(kosong karena tidak ada array)

=== DECORATED AST ===
└ ProgramNode(name: 'TesKosong') → tab_index:33, lev:0
  └ Block → block_index:1, lev:1
```

H. Pengujian 8

Input

```
program UndeclaredIdentifier;

var
  a: integer;

begin
  a := 5;
  b := a + 1;
```

```
end.
```

Output

```
=== SEMANTIC ERRORS ===
Semantic Error: Undeclared identifier: b

=== SYMBOL TABLE (tab) ===
idx  id              obj              type  ref  nrm  lev  adr  link
-----
...  (reserved words 0-32)
33   UndeclaredIdentifierprogram      0    0    1    0    0    0
34   a                  variable      1    0    1    0    0    33

=== BLOCK TABLE (btab) ===
idx  last  lpar  psze  vsze
-----
0     34    0     0     1      <- global block (program)
1      0    0     0     0

=== ARRAY TABLE (atab) ===
(kosong karena tidak ada array)

=== DECORATED AST ===
└ ProgramNode(name: 'UndeclaredIdentifier') → tab_index:33, lev:0
  └ Declarations
    └ VarDecl('a') → tab_index:34, type:integer, lev:0
      └ Block → block_index:1, lev:1
        └ Assign('a' := 5) → type:void
          └ target 'a' → tab_index:34, type:integer
            └ value 5 → type:integer
          └ Assign('b' := a+1) → type:void
            └ target 'b' → tab_index:-1, type:unknown
              └ value
                └ BinOp '+' → type:integer
                  └ 'a' → tab_index:34, type:integer
                    └ 1 → type:integer
```

I. Pengujian 9

Input

```
program TypeMismatchAssignment;

var
  intVar: integer;
  boolVar: boolean;

begin
  intVar := 42;
```

```

boolVar := true;
intVar := boolVar;
end.

```

Output

```

=== SEMANTIC ERRORS ===
Semantic Error: Type mismatch in assignment: cannot assign boolean to
integer

=== SYMBOL TABLE (tab) ===
idx  id              obj              type  ref  nrm  lev  adr  link
-----
...  (reserved words 0-32)
33   TypeMismatchAssignment  program    0    0    1    0    0    0
34   intVar                 variable   1    0    1    0    0    33
35   boolVar                variable   3    0    1    0    1    34

=== BLOCK TABLE (btab) ===
idx  last  lpar  psze  vsze
-----
0     35    0     0     2      <- global block (program)
1      0    0     0     0

=== ARRAY TABLE (atab) ===
(kosong karena tidak ada array)

=== DECORATED AST ===
└─ ProgramNode(name: 'TypeMismatchAssignment') → tab_index:33, lev:0
    └─ Declarations
        └─ VarDecl('intVar') → tab_index:34, type:integer, lev:0
            └─ VarDecl('boolVar') → tab_index:35, type:boolean, lev:0
        └─ Block → block_index:1, lev:1
            └─ Assign('intVar' := 42) → type:void
                └─ target 'intVar' → tab_index:34, type:integer
                    └─ value 42 → type:integer
            └─ Assign('boolVar' := true) → type:void
                └─ target 'boolVar' → tab_index:35, type:boolean
                    └─ value 'true' → tab_index:31, type:boolean
            └─ Assign('intVar' := boolVar) → type:void
                └─ target 'intVar' → tab_index:34, type:integer
                    └─ value 'boolVar' → tab_index:35, type:boolean

```

J. Pengujian 10

Input

```

program IncompatibleOperands;

var

```

```

x: integer;
flag: boolean;

begin
  x := 10;
  flag := true;
  x := x + flag;
end.

```

Output

```

=== SEMANTIC ERRORS ===
Semantic Error: Invalid types for operator plus: integer and boolean

=== SYMBOL TABLE (tab) ===
idx  id          obj          type  ref  nrm  lev  adr  link
-----
...  (reserved words 0-32)
33   IncompatibleOperandsprogram    0    0    1    0    0    0
34   x                variable    1    0    1    0    0    33
35   flag             variable    3    0    1    0    1    34

=== BLOCK TABLE (btab) ===
idx  last  lpar  psze  vsze
-----
0     35    0     0     2      <- global block (program)
1      0    0     0     0

=== ARRAY TABLE (atab) ===
(kosong karena tidak ada array)

=== DECORATED AST ===
└ ProgramNode(name: 'IncompatibleOperands') → tab_index:33, lev:0
  └ Declarations
    └ VarDecl('x') → tab_index:34, type:integer, lev:0
    └ VarDecl('flag') → tab_index:35, type:boolean, lev:0
    └ Block → block_index:1, lev:1
      └ Assign('x' := 10) → type:void
        └ target 'x' → tab_index:34, type:integer
        └ value 10 → type:integer
      └ Assign('flag' := true) → type:void
        └ target 'flag' → tab_index:35, type:boolean
        └ value 'true' → tab_index:31, type:boolean
      └ Assign('x' := x+flag) → type:void
        └ target 'x' → tab_index:34, type:integer
        └ value
          └ BinOp '+' → type:unknown
            └ 'x' → tab_index:34, type:integer
            └ 'flag' → tab_index:35, type:boolean

```

K. Pengujian 11

Input

```
program SubrangeBoundError;

var
  range: 10..5;

begin
  range := 7;
end.
```

Output

```
=== SEMANTIC ERRORS ===
Semantic Error: Subrange: lower bound cannot be greater than upper bound

=== SYMBOL TABLE (tab) ===
idx  id              obj              type  ref  nrm  lev  adr  link
-----
...  (reserved words 0-32)
33   SubrangeBoundErrorprogram    0    0    1    0    0    0
34   range                variable    0    0    1    0    0    33

=== BLOCK TABLE (btab) ===
idx  last  lpar  psze  vsze
-----
0     34    0     0     1      <- global block (program)
1     0     0     0     0

=== ARRAY TABLE (atab) ===
(kosong karena tidak ada array)

=== DECORATED AST ===
└ ProgramNode(name: 'SubrangeBoundError') → tab_index:33, lev:0
  └ Declarations
    └ VarDecl('range') → tab_index:34, type:unknown, lev:0
  └ Block → block_index:1, lev:1
    └ Assign('range' := 7) → type:void
      └ target 'range' → tab_index:34, type:unknown
        └ value 7 → type:integer
```

L. Pengujian 12

Input

```
program ArrayIndexTypeError;

var
```

```

arr: array[1..10] of integer;
index: boolean;

begin
  index := true;
  arr[index] := 42;
end.

```

Output

```

=== SEMANTIC ERRORS ===
Semantic Error: Array index must be simple type, got: unknown

=== SYMBOL TABLE (tab) ===
idx  id          obj          type  ref  nrm  lev  adr  link
-----
...  (reserved words 0-32)
33   ArrayIndexTypeErrorprogram    0    0    1    0    0    0
34   arr          variable    0    0    1    0    0    33
35   index        variable    3    0    1    0    1    34

=== BLOCK TABLE (btab) ===
idx  last  lpar  psze  vsze
-----
0    35    0     0     2      <- global block (program)
1     0    0     0     0

=== ARRAY TABLE (atab) ===
idx  xtyp  etyp  eref  low  high  elsz  size
-----
0     1    1     0    0    0     1    1

=== DECORATED AST ===
└ ProgramNode(name: 'ArrayIndexTypeError') → tab_index:33, lev:0
  └ Declarations
    └ VarDecl('arr') → tab_index:34, type:array, lev:0
    └ VarDecl('index') → tab_index:35, type:boolean, lev:0
    └ Block → block_index:1, lev:1
      └ Assign('index' := true) → type:void
        └ target 'index' → tab_index:35, type:boolean
        └ value 'true' → tab_index:31, type:boolean
      └ Assign('arr' := 42) → type:void
        └ target 'arr' → tab_index:34, type:integer
        └ value 42 → type:integer

```


IV. Kesimpulan dan Saran

A. Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, semantic analysis pada compiler Arion berhasil dikembangkan menggunakan pendekatan Abstract Syntax Tree (AST), Decorated AST, serta algoritma L-Attributed Grammar. Sistem mampu melakukan berbagai validasi semantik seperti type checking, scope resolution, symbol table management, dan validasi struktur kontrol program dengan baik. Selain itu, AST yang dibangun berhasil merepresentasikan struktur program secara lebih ringkas dan bermakna dibandingkan parse tree sehingga mempermudah proses analisis semantik. Hasil pengujian menunjukkan bahwa compiler dapat menangani berbagai fitur bahasa pemrograman seperti deklarasi variabel, prosedur, fungsi, operasi ekspresi, array, record, percabangan, serta perulangan tanpa menghasilkan error semantik pada kasus yang valid. Dengan demikian, milestone ini berhasil memenuhi tujuan utama dalam membangun semantic analyzer yang mampu menghasilkan decorated AST secara konsisten dan terstruktur.

B. Saran

Untuk pengembangan selanjutnya, semantic analyzer dapat ditingkatkan dengan menambahkan dukungan terhadap lebih banyak aturan semantik dan tipe data kompleks agar compiler menjadi lebih lengkap. Selain itu, mekanisme error handling dapat diperbaiki agar mampu memberikan pesan kesalahan yang lebih informatif, spesifik, dan mudah dipahami pengguna. Pengembangan optimasi pada struktur symbol table dan AST juga dapat dilakukan untuk meningkatkan efisiensi analisis pada program berskala besar. Di samping itu, fitur semantic checking tambahan seperti implicit type conversion, parameter compatibility checking yang lebih mendalam, serta validasi return function yang lebih ketat dapat menjadi fokus pengembangan berikutnya.

Referensi

Gambar Parse Tree dan Abstract Syntax Tree (AST)

<https://ruslanspivak.com/lrbasi-part7>

Penjelasan AST

<https://nus-cs3203.github.io/course-website/contents/basic-spa-requirements/abstract-syntax-tree.html>

“Semantic Analysis (ULiège)”

<https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>

Lampiran A. Link Penting

Github : [BST-Tubes-IF2224-2026](https://github.com/BST-Tubes-IF2224-2026)

Pembagian Tugas :

NIM	Persentase
13524019	25%
13524033	25%
13524043	25%
13524101	25%